Task 2 Report
Evaluating Chaotic Systems

System analysed: Lorenz system

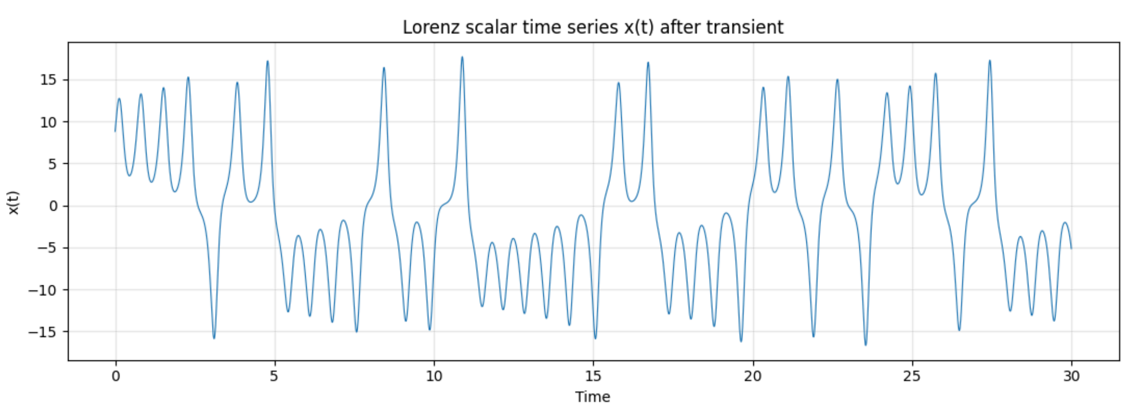
Part	Method	Main output
A	Phase Space Reconstruction (AMI + FNN)	tau = 16, m = 3
B	Lyapunov exponent	ODE spectrum and scalar time-series LLE
D	Correlation dimension	D2 saturation estimate approx. 1.9573
C	K2 entropy	Final K2 estimate approx. 2.1662
E	Box-counting dimension	D0 estimate approx. 2.1041

1. Lorenz System and Data Generation

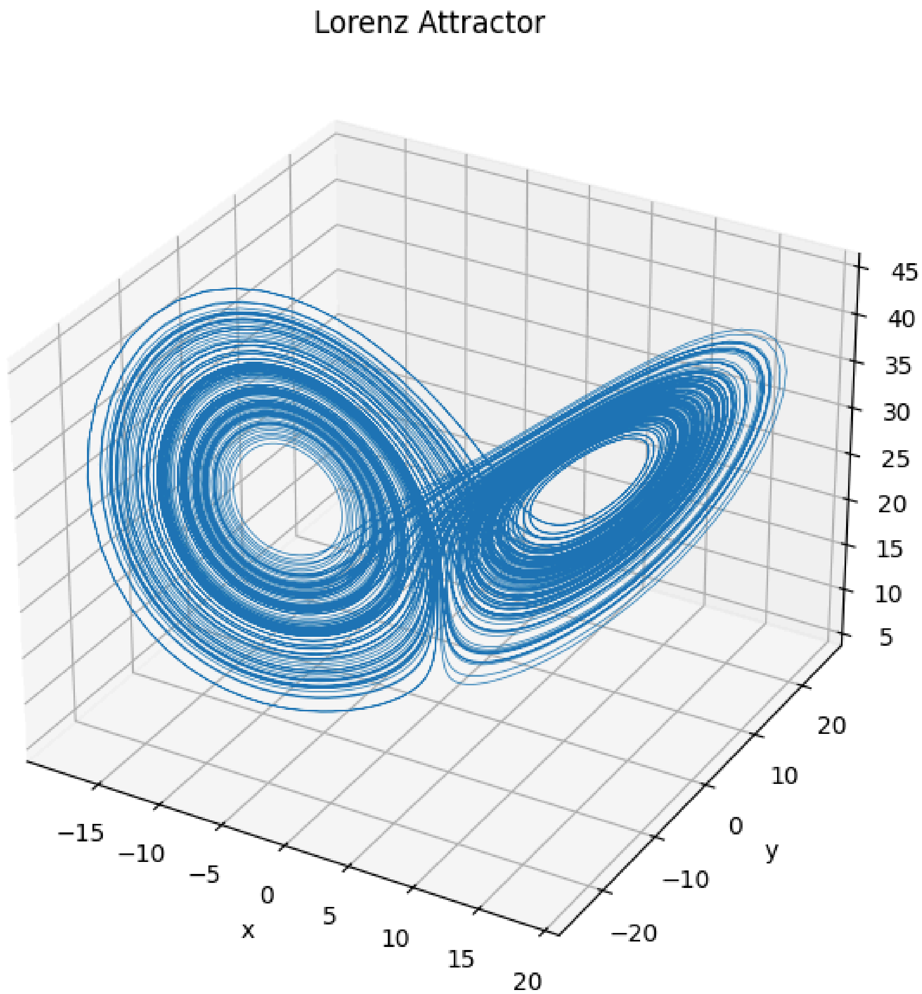
```
dx/dt = sigma * (y - x)
dy/dt = x * (rho - z) - y
dz/dt = x * y - beta * z
```

Setting	Value
sigma	10.0
rho	28.0
beta	8 / 3
Initial condition	[1.0, 1.0, 1.0]
Time step dt	0.01
Transient steps removed	5000
Kept steps after transient	15000
Scalar series used	x(t)

I removed the transient part because the first part of the simulation may depend too much on the initial condition. After removing it, the trajectory is already on the attractor.



This x(t) signal is important because later methods, such as PSR and time-series Lyapunov, work using only one scalar signal. The signal looks irregular and does not repeat in a simple periodic way.



The 3D attractor shows the well-known two-wing shape of Lorenz. The trajectory stays bounded, but it moves in a complex way. This is why it is considered chaotic.

2. Part A - Phase Space Reconstruction (PSR)

Phase Space Reconstruction is used when we only have one measured signal, for example x(t), but we want to rebuild the hidden attractor shape. Instead of using x, y, and z directly, we build new vectors from delayed copies of x(t).

```
X_i = [x_i, x_(i + tau), x_(i + 2*tau), ..., x_(i + (m-1)*tau)]
```

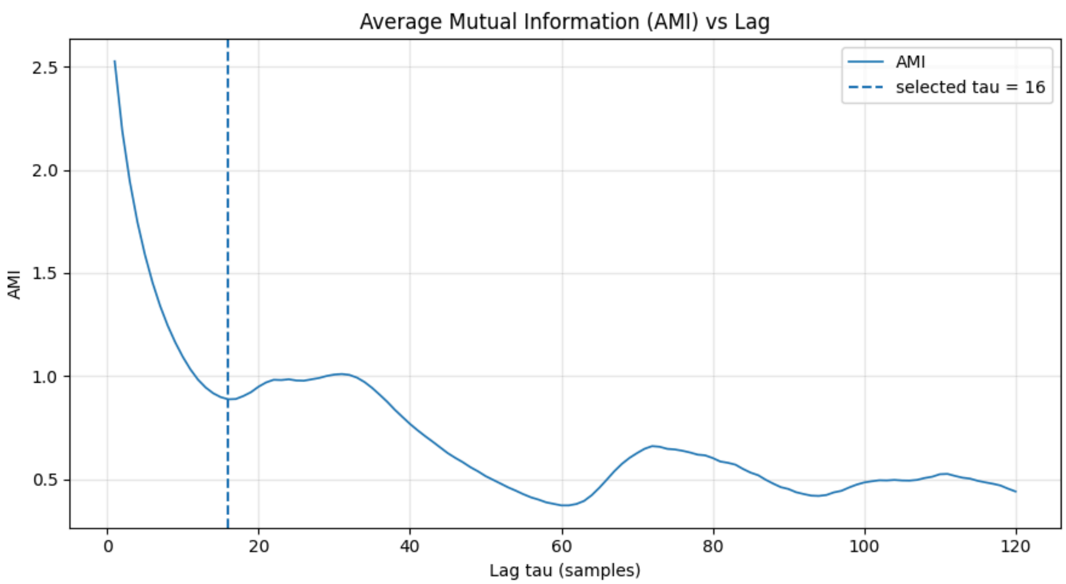
The two important choices are tau and m. The delay tau decides how far apart the copied values are. The embedding dimension m decides how many delayed coordinates we use.

Parameter	Meaning	Value used
tau	Delay between copied values of x(t)	Estimated using AMI
m	Number of delayed coordinates	Estimated using FNN
Input signal	One scalar time series	x(t)

2.1 Time Delay using AMI

Average Mutual Information checks how much information x(t) and a delayed version x(t + tau) still share. If tau is too small, the delayed values are almost the same. If tau is too large, the connection may become weak. So we choose the first local minimum.

AMI setting	Value
max_lag	120
n_bins	32
criterion	first local minimum
standardize	True
Selected tau	16 samples

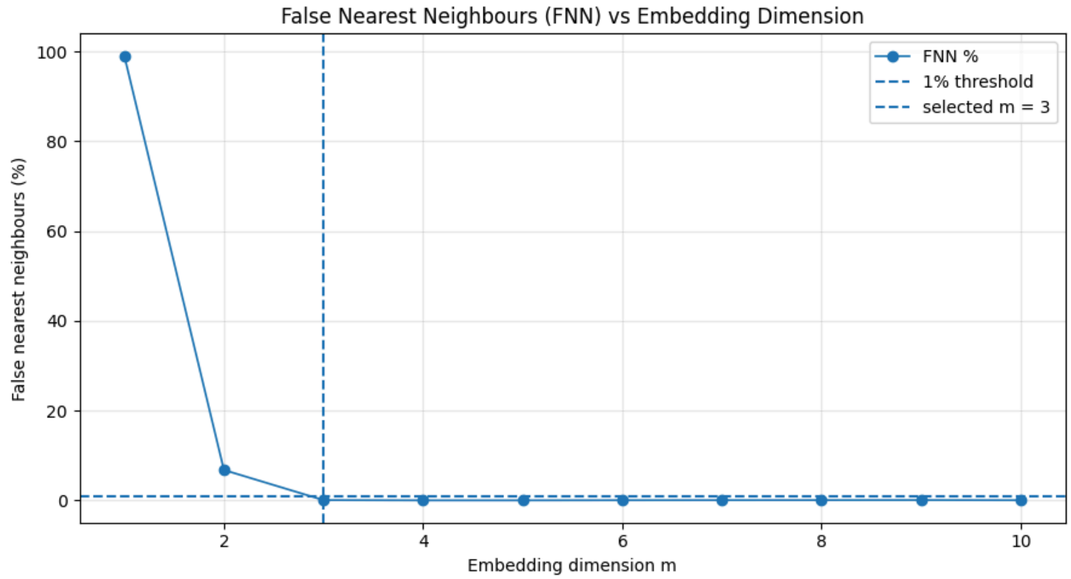


The selected delay was tau = 16. This value is also inside the expected range for Lorenz, which was around 10 to 17 samples. I understand this as a good delay because it keeps useful information without making the coordinates too similar.

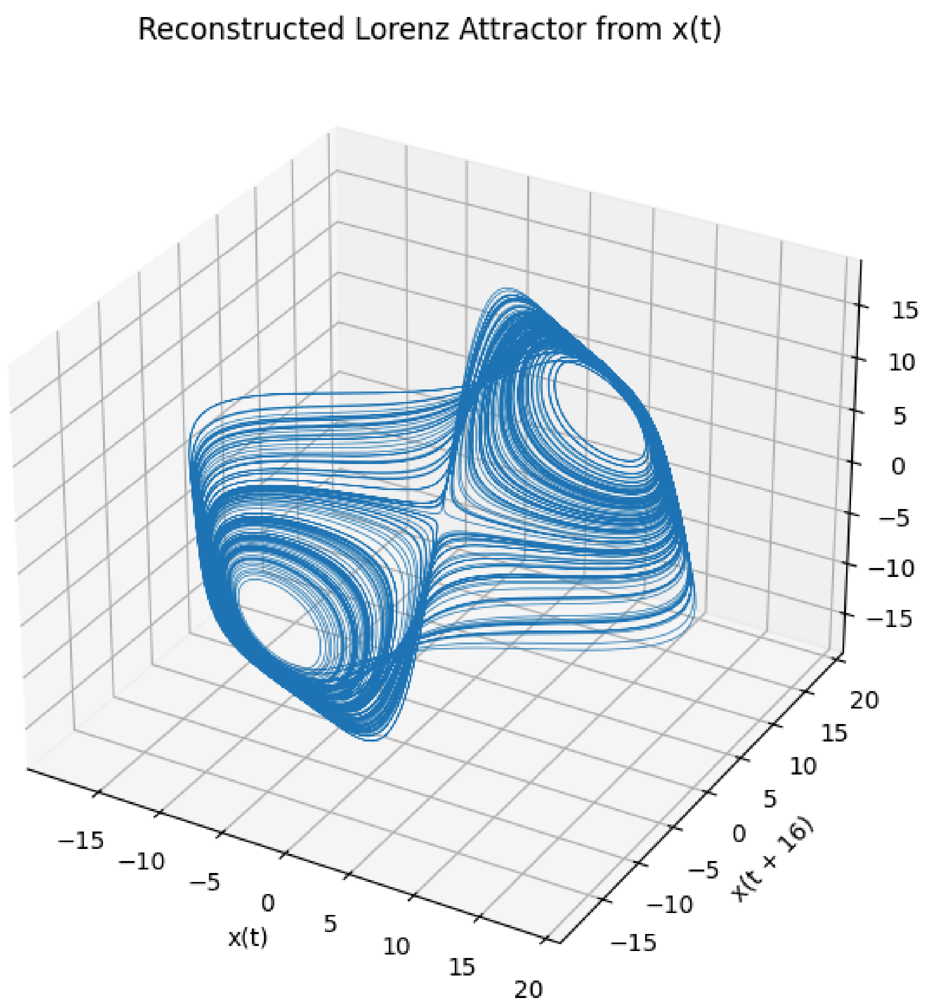
2.2 Embedding Dimension using FNN

False Nearest Neighbours is used to find the needed embedding dimension. If the dimension is too small, points may look close only because the attractor is folded in projection. When we increase the dimension, these false neighbours should disappear.

FNN setting	Value
max_dim	10
R_tol	10.0
A_tol	2.0
threshold_percent	1.0%
Theiler window	50
Selected m	3



The FNN percentage drops below the 1% threshold at m = 3. This means that three delayed coordinates are enough for reconstruction. This also agrees with the original Lorenz system because it has three state variables: x, y, and z.



The reconstructed attractor does not look exactly like the original x, y, z attractor. But the important point is that it still captures the same underlying structure, especially the two main circular regions, using only x(t).

Quantity	Result
Selected delay tau	16
Selected embedding dimension m	3
Comparison to Lorenz dimension	m = 3 matches the original 3D state space

3. Part B - Lyapunov Exponent

The Lyapunov exponent measures how fast two very close trajectories separate. If the largest Lyapunov exponent is positive, this means small differences grow with time. This is one of the main signs of chaos.

In this task I used two methods: one from the known ODE equations and one from the scalar time series x(t). The ODE method is more accurate because it uses the equations while time series uses the reconstructed phase space made earlier.

ODE Lyapunov setting	Value
Method	Wolf Gram-Schmidt reorthogonalisation
dt	0.01
Transient steps	2000
n_steps	30000
Solver	Fixed-step RK4
ortho_steps	20
log_base	e for nats/time

Lyapunov spectrum from ODE	nats/time	bits/time
lambda_1	0.89750	1.29483
lambda_2	-0.00264	-0.00380
lambda_3	-14.56143	-21.00771

The largest exponent is positive, lambda_1 = 0.89750 nats/time. This confirms chaotic behavior. The second exponent is very close to zero, which is normal for a continuous flow. The third exponent is negative, meaning there is contraction in another direction.

Time-series Lyapunov setting	Value
Input signal	x(t)
tau	16
m	3
theiler_window	50
min_dist_scale	1e-3
max_dist_scale	1e-1
evolve_cap	50
max_replacements	200
angle_weight	0.3

Method	lambda_1	Units	Notes
ODE Wolf-GSR	1.29483	bits/time	Full spectrum method using equations and Jacobian
Time-series Wolf	2.41989	bits/time	Uses scalar x(t), PSR, and neighbour tracking

The two lambda_1 values are not exactly the same. This is expected because the time-series method is an approximation and depends on tau, m, neighbour selection, and finite data length. The important result is that both methods give a positive largest exponent, so both support that the Lorenz system is chaotic.

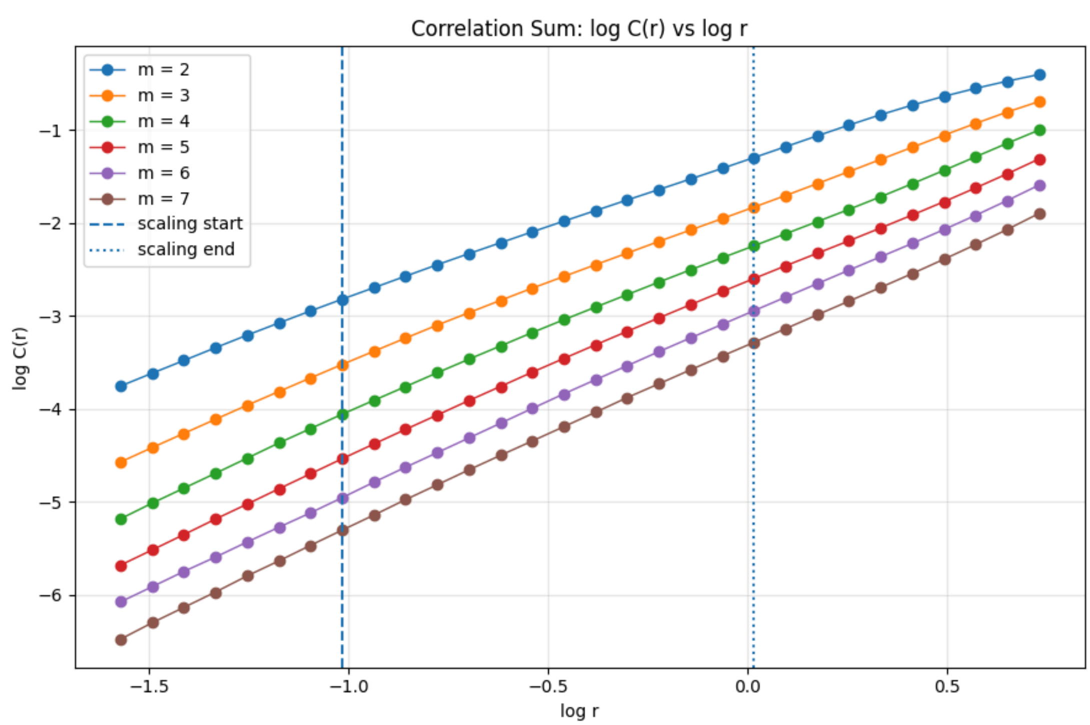
Debug information from the time-series method: tracked segments = 200 and physical time used = 99.78. This means the method had enough tracked segments to compute an estimate, but the estimate can still be sensitive to settings.

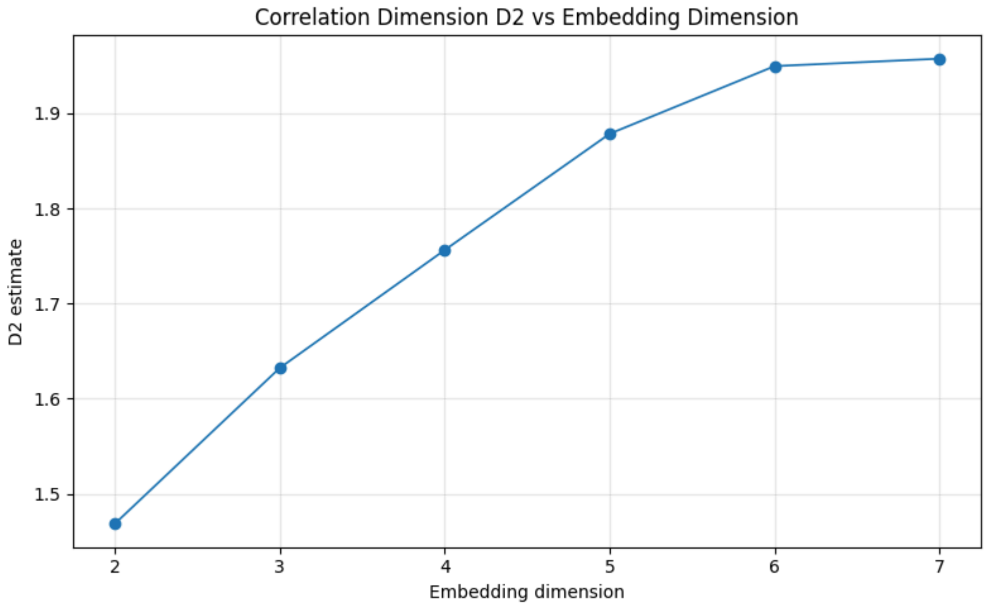
4. Part D - Correlation Dimension

I did Part D before Part C because the K2 entropy uses the correlation sums calculated here.

Correlation dimension estimates how the points of the attractor fill space. We count the fraction of point pairs closer than a radius r. If log C(r) versus log r has a linear region, the slope gives an estimate of the correlation dimension D2.

Correlation dimension setting	Value
Embedding dimensions tested	2 to 7
tau	16
Radii count	30
r_min	0.20762
r_max	2.07454
Scaling region indices	7 to 20
Max points used	2200
Theiler window	max(50, tau * dimension)





Embedding dim.	D2 estimate	Points used	Valid pairs	Theiler window
2	1.46845	2200	2403528	50
3	1.63224	2200	2403528	50
4	1.75620	2200	2399145	64
5	1.87838	2200	2394766	80
6	1.94949	2200	2388205	96
7	1.95733	2200	2383836	112

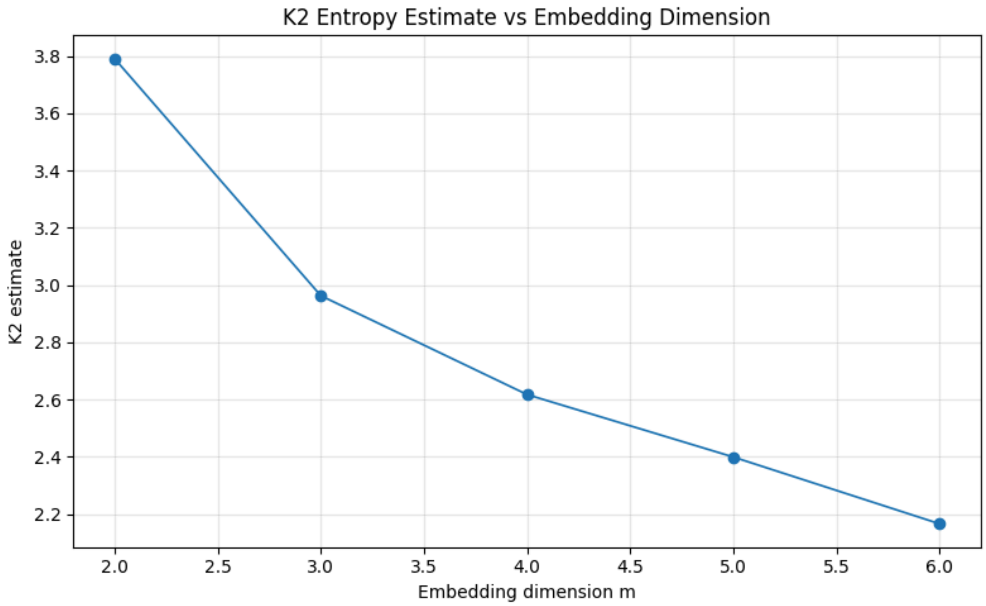
The correlation dimension increases at first as the embedding dimension increases. After that, it starts to change more slowly and reaches about D2 = 1.9573. This kind of saturation suggests that the Lorenz attractor is low-dimensional and fractal, not random high-dimensional noise.

5. Part C - Kolmogorov K2 Entropy

K2 entropy estimates how fast information is produced, or how fast predictability is lost. In simple words, a larger positive K2 means the system becomes harder to predict faster.

Here K2 was estimated using the correlation sums from Part D for consecutive embedding dimensions m and m+1.

K2 setting	Value
Input	Correlation sums from Part D
Dimension pairs	2->3, 3->4, 4->5, 5->6, 6->7
Scaling region	Same as Part D
Formula idea	log(C_m / C_(m+1)) / (tau * dt)
tau * dt	16 * 0.01 = 0.16



m to m+1	m	K2 estimate	Radii used
2 to 3	2	3.79090	14
3 to 4	3	2.96293	14
4 to 5	4	2.61774	14
5 to 6	5	2.39954	14
6 to 7	6	2.16623	14

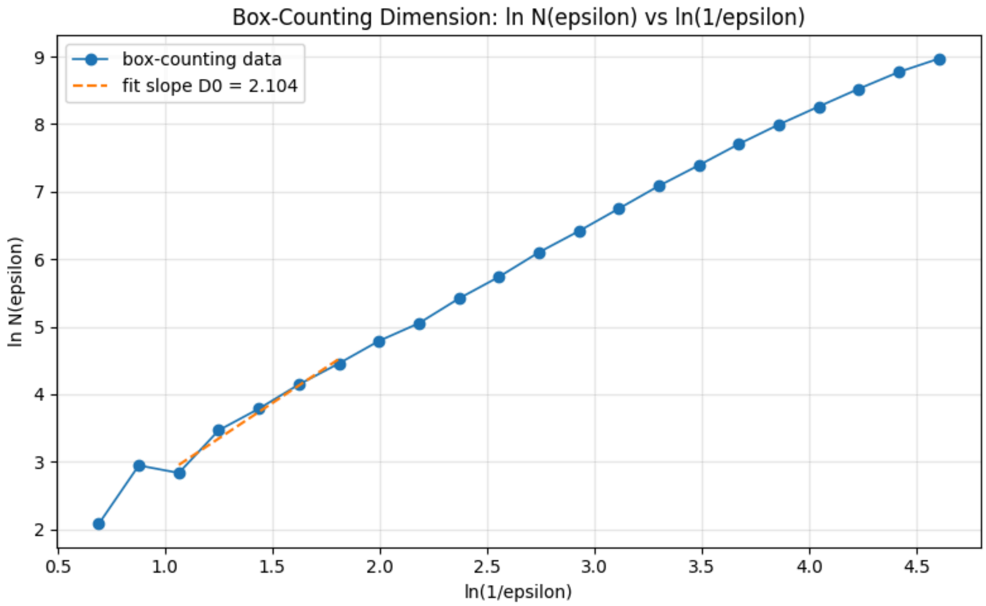
The K2 estimate decreases and moves toward a finite positive value. The final estimate is about K2 = 2.1662. Since it does not grow without bound, this supports the idea that the system is low-dimensional chaos rather than random noise.

6. Part E - Fractal / Box-Counting Dimension

Box-counting dimension is another way to estimate fractal dimension. We divide the phase space into boxes of size epsilon and count how many boxes contain at least one point from the attractor. This count is called N(epsilon).

If the attractor fills more space, more boxes are needed. The slope of ln N(epsilon) versus ln(1/epsilon) gives the box-counting dimension D0.

Box-counting setting	Value
Input points	Original 3D Lorenz attractor after transient
Normalization	Each coordinate normalized to [0, 1]
epsilon range	0.5 to 0.01
Number of epsilon values	22
Scaling region indices	2 to 6



Measure	Estimate	Meaning
Correlation dimension D2	1.95733	Based on pair distances
Box-counting dimension D0	2.10410	Based on occupied boxes

The estimated box-counting dimension is D0 = 2.1041. It is slightly larger than D2 = 1.9573, which is acceptable because box-counting and correlation dimension are related but not exactly the same. Both values are close to 2, which means the attractor is more complex than a simple curve but does not fill the whole 3D space.

7. Final Conclusion

In this task, I evaluated the Lorenz system using several quantitative methods. The PSR result gave tau = 16 and m = 3, which means the attractor can be reconstructed from only x(t). The Lyapunov result showed a positive largest exponent, which confirms sensitive dependence on initial conditions.

The correlation dimension and box-counting dimension were both around 2. This means the Lorenz attractor has a fractal structure. The K2 entropy was finite and positive, which also supports low-dimensional chaotic behavior. Overall, all results agree that the Lorenz system is chaotic and has a strange/fractal attractor.

Quantity	Final value	Interpretation
tau	16	Useful delay for reconstructing x(t)
m	3	Enough embedding dimension for Lorenz reconstruction
ODE lambda_1	0.89750 nats/time	Positive, so the system is chaotic
Time-series lambda_1	2.41989 bits/time	Also positive, but more sensitive to settings
D2	1.95733	Low-dimensional fractal estimate
K2	2.16623	Finite positive entropy estimate
D0	2.10410	Box-counting fractal dimension

Appendix - Code Used

The full code is available in the submitted Jupyter notebook. The notebook uses the provided psr.py file for AMI and FNN, and the provided lyapunov.py file for Lyapunov exponent estimation. The remaining code in the notebook calculates the correlation dimension, K2 entropy, and box-counting dimension.

Code files used:

- psr.py: estimate_delay_ami, estimate_dimension_fnn, reconstruct_matrix
- Lyapynov.py: lyapunov_wolf_ode, wolf_mle
- Task 2 notebook: simulation, plots, tables, and final numerical estimates